

실습 : MiniC #2

프로그래밍언어개론

정민경 (jmink002@gmail.com)

MiniC

- C의 부분집합
 - ▶ 변수 선언 및 할당
 - ▶ 분기문
 - ▶ 반복문
 - ▶ 포인터
 - ▶ 함수

기존 Store 대신
포인터 지원을 위한 추상메모리 모델 Env X Mem
+ 메모리 주소를 값으로 정의 (Value)

MiniC

- C의 부분집합
 - ▶ 변수 선언 및 할당
 - ▶ 분기문
 - ▶ 반복문
 - ▶ 포인터

▶ 함수

함수 정보 저장을 위한 Fstore 정의

MiniC AST

expression

```
type expr =  
  | Num of int  
  | Ref of string  
  | Id of string  
  | Bool of bool  
  | Add of expr * expr  
  | Sub of expr * expr  
  | Lt of expr * expr  
  | Gt of expr * expr  
  | Eq of expr * expr  
  | And of expr * expr  
  | Or of expr * expr
```

(* parser/ast.ml *)

MiniC AST

expression

- ...
- Ref: 변수의 주소값 구하는 연산을 나타내는 AST 노드 (&x)
- ...

MiniC AST

statement

```
type stmt =  
  | DefStmt of string * expr  
  | StoreStmt of expr * expr  
  | LoadStmt of string * expr  
  | IfStmt of expr * stmt list * stmt list  
  | LoopStmt of expr * stmt list  
  | ReturnStmt of expr  
  | CallStmt of string * string * expr list
```

(* parser/ast.ml *)

MiniC AST

statement

- DefStmt: 변수 선언을 수행하는 AST 노드 (변수 선언 시 항상 초기화 진행)
- StoreStmt: 첫 expr 연산 결과 주소에 두번째 expr 연산 결과 값을 저장하는 AST 노드
- LoadStmt: expr 연산 결과 주소에서 값을 꺼내와 변수에 저장하는 AST 노드
- IfStmt: 첫 expr 연산 결과에 따라 stmt list를 수행하는 AST 노드
- LoopStmt: 첫 expr 연산 결과가 참일 동안 stmt list 수행을 반복하는 AST 노드
- ReturnStmt: expr 연산 결과 값을 메모리 내 반환값 주소에 저장하는 AST 노드
- CallStmt: expr list 연산 결과 값을 인자로 함수 호출 후, 그 결과를 저장하는 AST 노드

MiniC AST

function definition & program

```
type def =  
  | FunDef of string * string list * stmt list  
  
type prog = Program of def list * stmt list
```

(* parser/ast.ml *)

- FunDef: 함수 정의 AST 노드
- Program: MiniC 프로그램 AST 노드

과제 : MiniC

hw12.ml

- 아래 함수들을 작성하여 MiniC interpreter를 구현하시오
- `interp_prog: Ast.prog -> (Env.t * Mem.t)`
- `interp_fundef: Ast.def -> Fstore.t -> Fstore.t`
- `interp_stmt: Ast.stmt -> Fstore.t -> (Env.t * Mem.t) -> (Env.t * Mem.t)`
- `interp_expr: Ast.expr -> (Env.t * Mem.t) -> Value.t`

과제 : MiniC

env.ml

- add는 항상 앞에 추가하며, 중복 변수가 있는 경우 제거 후 추가
- find는 찾을 변수가 존재하지 않을 경우, Free identifier: <변수>

```
type addr = int
type t = (string * addr) list
let empty : t = []
val add : string -> addr -> t -> t
val find : string -> t -> addr
```

(* env.ml *)

과제 : MiniC

mem.ml

- add는 항상 앞에 추가하며, 중복 주소가 있는 경우 제거 후 추가
- find는 찾을 주소가 존재하지 않을 경우, Free address: <주소>

```
type t = (Env.addr * Value.t) list

let empty : t = []

val add : Env.addr -> Value.t -> t -> t
val find : Env.addr -> t -> Value.t
```

(* mem.ml *)

과제 : MiniC

추상메모리

- 추상메모리 = Env.t * Mem.t

```
def w;
def x;
def y;
def z;
*&x = 3;
*&y = &x;
*&z = &y;
*y = 4;
if(x > 3){
    w = *z;
    *w = x - 1;
}
```



w x y z

$\sigma : Var \rightarrow Addr$



a0 a1 a2 a3 a4 a5 a6

$m : Addr \rightarrow Value$

과제 : MiniC

fstore.ml

- add는 항상 앞에 추가하며, 중복 함수 이름이 있는 경우 제거 후 추가
- find는 찾을 함수가 존재하지 않을 경우, Unbound function: <함수>

```
type t = (string * string list * Ast.stmt list) list

let empty : t = []

val add : string -> string list -> Ast.stmt list -> t -> t
val find : string -> t -> (string list * Ast.stmt list)
```

(* fstore.ml *)

과제 : MiniC

value.ml

- NumV : 정수값을 나타내는 type constructor
- BoolV : 논리값을 나타내는 type constructor
- AddrV : 주소를 나타내는 type constructor

```
type t =  
  | NumV of int  
  | BoolV of bool  
  | AddrV of int
```

(* value.ml *)

과제 : MiniC

addrManager.ml

- 주소를 관리하는 모듈 AddrManager는 아래와 같이 정의
 - init: unit -> unit
 - 주소 관리 모듈 초기화
 - interp_prog 함수 시작시 항상 init 먼저 호출
 - new_addr: unit -> addr
 - 새 주소 반환 (초기화 시 주소는 0으로 초기화되며, 새 주소를 얻을 때 마다 1씩 증가)
 - ret_addr: addr
 - 반환 값을 저장할 주소 (-1로 setting 되어있음)

과제 : MiniC

Failwith in hw12.ml

- Add, Sub, Lt, Gt에서 `expr0`이 숫자가 아닌 값으로 계산되는 경우:
 - Not a number: 해당 expression 문자열
 - e.g. `Hw12.interp_prog (ParserMain.parse "def x = 1 + true;")`
 - Failure "Not a number: 1 + true"
- And, Or, IfStmt, LoopStmt에서 `expr0`이 논리 값이 아닌 값으로 계산되는 경우:
 - Not a bool: 해당 expression 문자열
 - e.g. `Hw12.interp_prog (ParserMain.parse "def x = 3; if x {x = x - 2;}")`
 - Failure "Not a bool: x"

과제 : MiniC

Failwith in hw12.ml

- Loadstmt, StoreStmt에서 expr 계산 결과가 주소 값이 아닌 경우
 - Not an address: 해당 expression 문자열
 - e.g. Hw12.interp_prog (ParserMain.parse “def x = 0; x = *1;”)
 - e.g. Hw12.interp_prog (ParserMain.parse “*1 = 2;”)
 - Failure “Not an address: 1”
- CallStmt에서 인자 개수가 다른 경우:
 - The number of arguments not matched: actual [인자 개수], expected [파라미터 개수]
 - e.g. Hw12.interp_prog (ParserMain.parse “fun f(a, b) { return a + b; } def x = 0; x = f(1, 2, 3);”)
 - Failure "The number of arguments not matched: actual 3, expected 2"

과제 : MiniC

hw12_test.ml

- hw12_test.ml에 hw12.ml에 대한 코드 커버리지를 100% 달성하는 테스트 작성
- code coverage 100% 달성하는 coverage report 스크린샷(test_coverage.png) 함께 제출
- 별도로 제공한 parser 폴더, addrManager.ml 모두 포함하여 제출
- 주어진 함수 타입은 반드시 준수! (함수 이름, 타입 오류로 인한 컴파일 에러 시 0점)

과제 : MiniC

Coverage Testing

- 테스트 수행
 - `dune runtest --instrument-with bisect_ppx --force`
 - `_build/default/bisect~~.coverage` 파일 생성 확인\
- Code Coverage report 생성
 - `bisect-ppx-report html`
- Code Coverage 확인
 - `_coverage/index.html` 확인

과제 제출

- 제출 기한 : **5월 29일 목요일 23:59:59 까지** (추가 제출 기한 없음)
- 제출 파일 : *.ml, dune, parser 폴더, hw12_test.ml, test_coverage.png
- 제출 형식 : 하나의 파일로 압축 후 제출 (압축파일명 : 학번.zip)
- 주의 사항
 - ▶ 명시된 제출 파일 이외 다른 파일 제출 금지 (dune clean 후 제출)
 - ▶ 압축 해제 시, 제출 파일이 바로 등장하도록 함 (폴더 내에 포함시키지 말 것)
 - ▶ **파일명, 함수 이름, 파라미터 개수 및 순서 반드시 준수**
 - ▶ 대소문자, 띄어쓰기와 같은 오타 전부 예외없이 감점
 - ▶ 컴파일 에러 발생 시 0점
 - ▶ ChatGPT 사용 금지

10주차 과제 점수 확인

- 점수에 이상이 있거나 궁금한 점이 있다면
- 매주 실습시간에 튜터에게 문의하시면 됩니다
- 실습시간 이후에는 과제 점수에 대한 문의, 클레임 받지 않습니다

Question?

- 수업 or 과제와 관련된 질문은 질문페이지를 통해 질문
 - ▶ <https://answer.cnu-ants.synology.me/tags/upl>
 - ▶ 프로그래밍언어개론 태그로 질문 추가
 - ▶ “N주차 질문”과 같은 모호한 제목 자체 (질문 내용을 한줄 요약하여 제목으로)
 - ▶ 질문을 공유합시다 :)

Question?

- 수업 or 과제가 아닌 개인적인 문의사항은 메일로
 - ▶ 00분반 TA : jmink002@gmail.com
 - ▶ 01분반 TA : sy.baek28@gmail.com
 - ▶ 메일 제목: [UPL25] 제목 + 이름